

Fundamentos de Computadores

1º curso del Grado en Ingeniería Informática

Tema 4

Bloques funcionales combinacionales



*Departamento de Ingeniería Electrónica,
de Sistemas Informáticos y Automática*

Tema 4. Bloques funcionales combinacionales

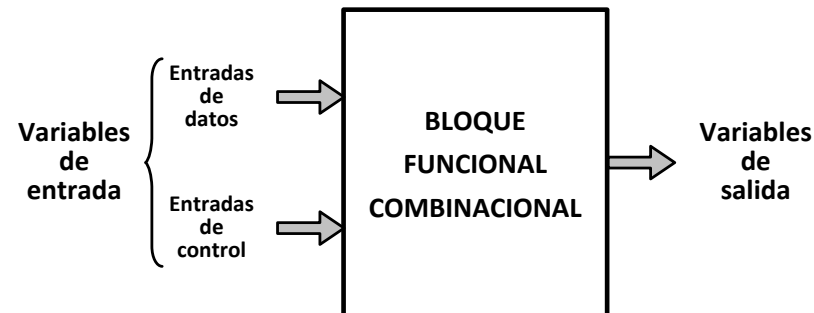
Introducción a los bloques funcionales combinacionales

En cualquier computador es preciso realizar operaciones de codificación y decodificación de señales digitales, operaciones aritméticas, multiplexado y demultiplexado, transmisiones y control de datos usando buses, etc.

Los fabricantes de circuitos integrados han desarrollado diversos bloques funcionales, cuyas primitivas fundamentales son las puertas lógicas estudiadas en los temas anteriores.

Estos bloques se engloban dentro de los dispositivos de media escala de integración o MSI (*Medium Scale Integration*) y poseen la ventaja de poder usarse como entidades de diseño de nivel superior a las puertas. De esta forma, no es necesario recordar la estructura interna de puertas de dichos bloques, sino que es suficiente con conocer sus símbolos y sus comportamientos.

Simbología general de los bloques funcionales combinacionales



En este tema estudiaremos los bloques funcionales combinacionales más frecuentemente utilizados y sus aplicaciones:

- **Decodificadores.**
- **Codificadores.**
- **Multiplexores.**
- **Demultiplexores.**
- **Comparadores.**
- **Detectores/generadores de paridad.**
- **Aplicaciones de los circuitos enumerados.**

Tema 4. Bloques funcionales combinacionales

Decodificadores

Un **decodificador** es un circuito lógico combinacional que posee **n** entradas de selección y **2ⁿ** salidas como máximo.

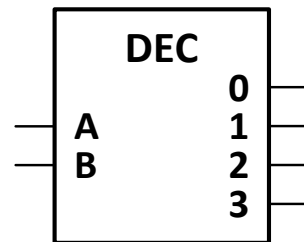
Su funcionamiento es tal que para cada combinación de las variables de entrada se activa una única salida, que es la que corresponde a la combinación binaria aplicada a las entradas.

Si el decodificador posee una salida para cada combinación de las variables de entrada (**2ⁿ** salidas) se denomina **decodificador completo**, y si no es así (menos de **2ⁿ** salidas) se denomina **decodificador incompleto**.

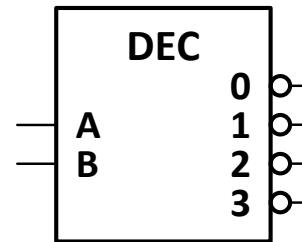
La principal aplicación de los decodificadores es la selección de un único dispositivo en cada momento (módulo de memoria, buffer triestado, etc.) entre todos los conectados a sus salidas, en función de la combinación aplicada a sus líneas de selección. No obstante, los decodificadores también pueden emplearse para implementar funciones lógicas sin necesidad de simplificarlas.

Los decodificadores pueden tener salidas activas a nivel alto (salida inactiva igual a 0 y salida activa igual a 1) o salidas activas a nivel bajo (salida inactiva igual a 1 y salida activa igual a 0).

Decodificadores de 2 a 4 líneas con salidas directas y negadas



B	A	Y ₀	Y ₁	Y ₂	Y ₃
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1



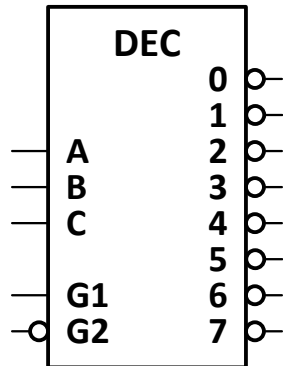
B	A	Y ₀	Y ₁	Y ₂	Y ₃
0	0	0	1	1	1
0	1	1	0	1	1
1	0	1	1	0	1
1	1	1	1	1	0

Tema 4. Bloques funcionales combinacionales

Decodificadores

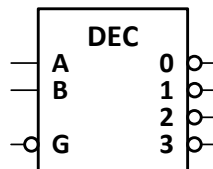
Entradas de habilitación

Algunos decodificadores poseen una o varias entradas de habilitación (**G**), las cuales deben encontrarse en un determinado estado lógico para que se produzca la decodificación. En caso contrario, el circuito no decodifica, es decir, no se selecciona ninguna salida.

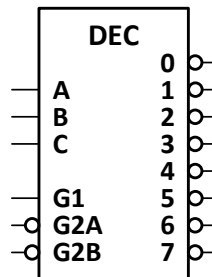


G ₂	G ₁	C	B	A	Y ₀	Y ₁	Y ₂	Y ₃	Y ₄	Y ₅	Y ₆	Y ₇
1	X	X	X	X	1	1	1	1	1	1	1	1
X	0	X	X	X	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	1	1	1
0	1	0	0	1	1	0	1	1	1	1	1	1
0	1	0	1	0	1	1	0	1	1	1	1	1
0	1	0	1	1	1	1	1	0	1	1	1	1
0	1	1	0	0	1	1	1	1	0	1	1	1
0	1	1	0	1	1	1	1	1	1	0	1	1
0	1	1	1	0	1	1	1	1	1	1	0	1
0	1	1	1	1	1	1	1	1	1	1	1	0

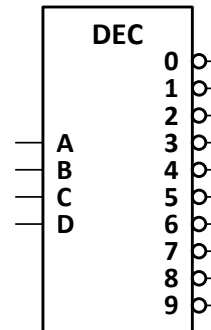
Decodificadores integrados comerciales



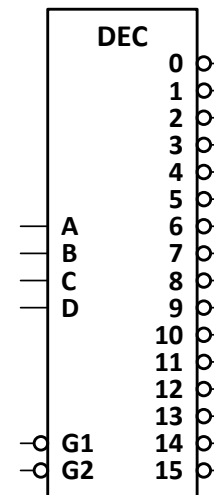
7439



74138



7442



74154

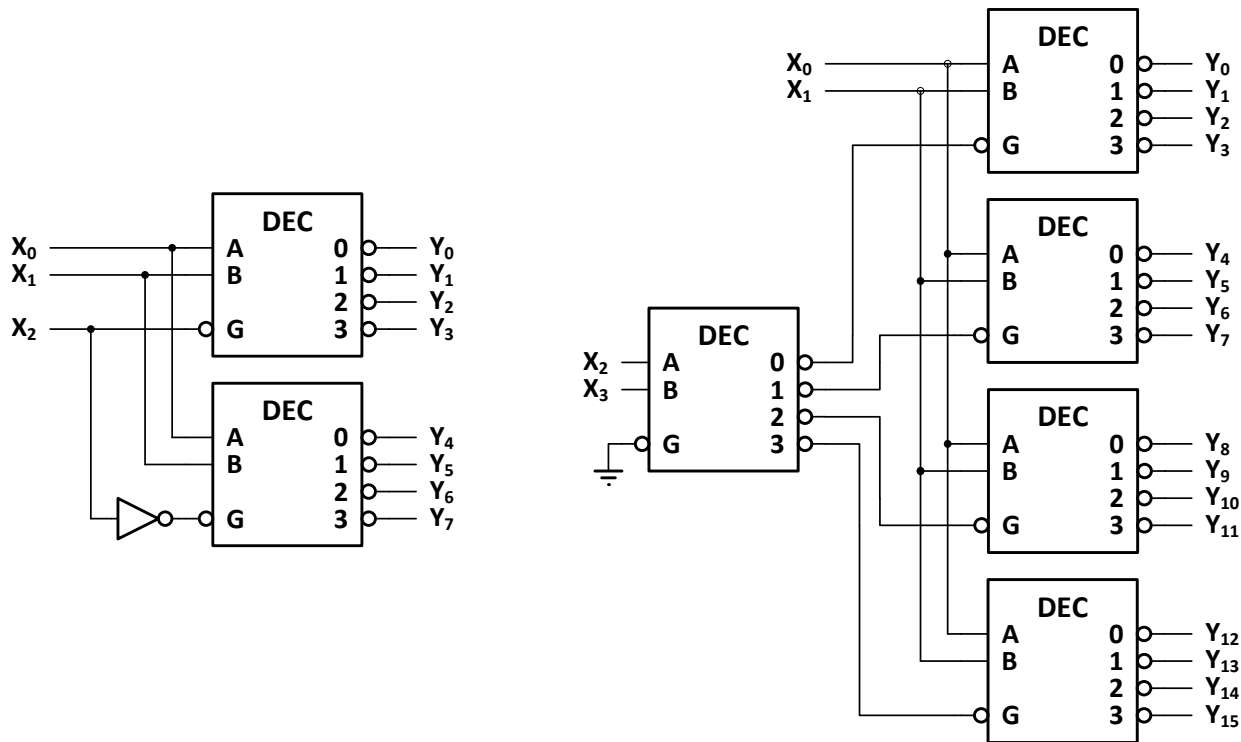
Tema 4. Bloques funcionales combinacionales

Decodificadores (Asociación de decodificadores)

Si se necesita obtener un decodificador con un tamaño diferente al de los disponibles comercialmente, se debe realizar una asociación de decodificadores. Para ello, se divide el número de salidas necesarias entre el número de salidas de los decodificadores de mayor tamaño disponibles, obteniendo así el número de decodificadores necesarios en la etapa de salida. A estos decodificadores se les aplicará en paralelo las variables de **menor peso** de la combinación de entrada.

Por último, es necesario que sólo uno de los decodificadores anteriores actúe en cada momento, cuando el valor binario de la combinación de entrada pertenezca al rango de salidas que proporciona ese decodificador. Esto se consigue añadiendo un nuevo decodificador (o puertas lógicas) para seleccionar uno entre los decodificadores de salida. A este último decodificador se le aplicarán las variables de **mayor peso** de la combinación de entrada.

Ejemplos: Obtener un decodificador de 3 a 8 líneas y otro de 4 a 16 líneas usando decodificadores de 2 a 4 líneas.



Tema 4. Bloques funcionales combinacionales

Decodificadores (Realización de funciones lógicas con decodificadores)

Un decodificador completo genera todos los minitérminos del conjunto de variables de entrada.

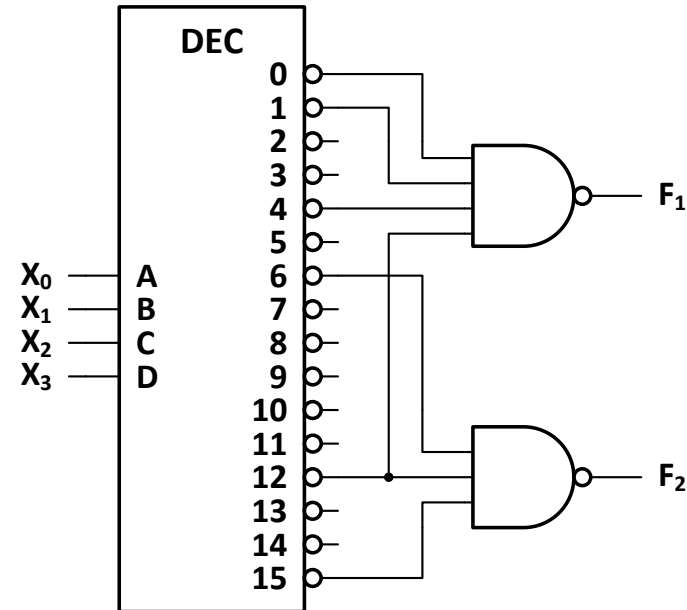
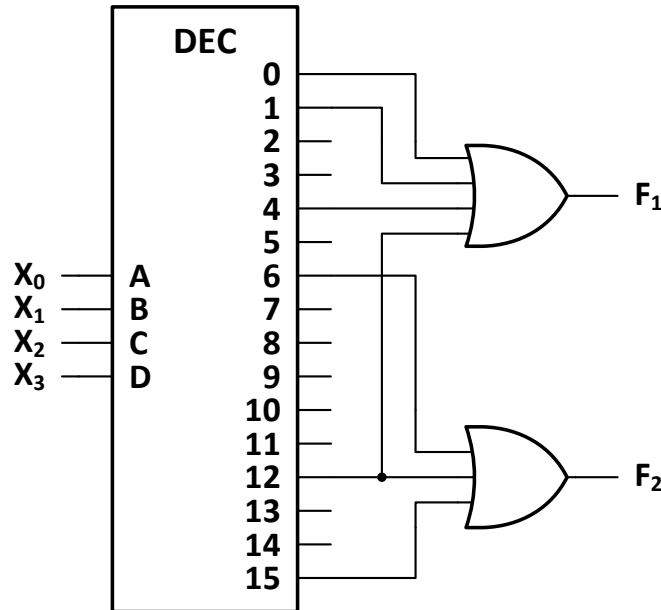
Así pues, para realizar funciones basándose en el empleo de un decodificador, basta con sumar (mediante el empleo de puertas **OR**) los términos pertenecientes a la expresión canónica disyuntiva numérica, en el caso de que las salidas del decodificador sean activas a **nivel alto**.

Si las salidas del decodificador son activas a **nivel bajo**, se procederá del mismo modo, excepto que se emplearán puertas **NAND** en lugar de puertas OR.

Ejemplo: Implementar mediante un decodificador las siguientes funciones:

$$F_1 = \sum_4(0, 1, 4, 12)$$

$$F_2 = \sum_4(6, 12, 15)$$

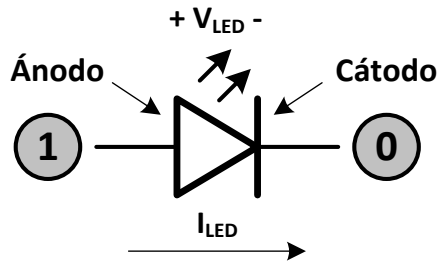


Tema 4. Bloques funcionales combinacionales

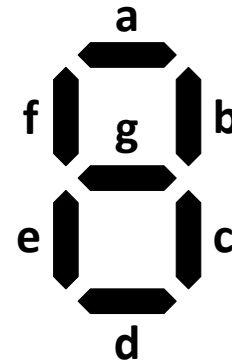
Decodificadores (Conversores de BCD a 7 segmentos)

Un conversor de BCD a 7 segmentos es un tipo especial de decodificador que recibe a su entrada una combinación expresada en código BCD natural y genera las señales necesarias para representar el dígito decimal correspondiente en un visualizador (display) de 7 segmentos. Posee salidas de tipo **driver** o **excitador**, capaces de proporcionar corrientes superiores a las de los circuitos estándar.

Un visualizador de siete segmentos es una estructura integrada por siete LEDs (*Light Emitter Diode*). Los LEDs son dispositivos que emiten luz cuando la corriente que los atraviesa excede de un cierto valor, de ahí que para su excitación se necesiten dispositivos que proporcionen corriente suficiente (drivers). Al igual que los diodos normales, los LEDs conducen cuando se polarizan de forma directa con una tensión que supere un determinado valor.

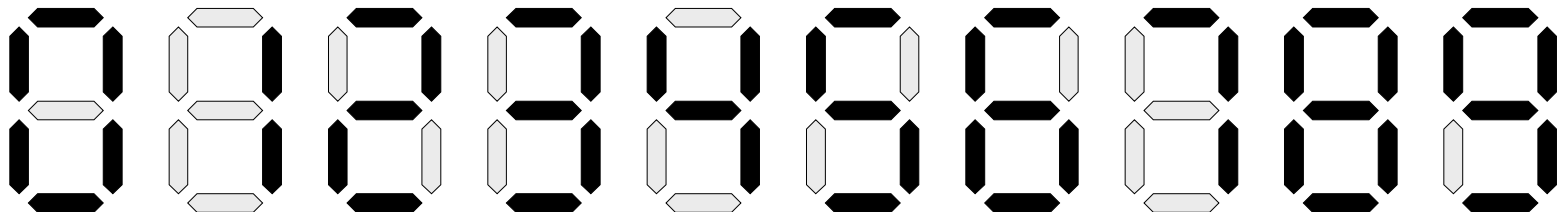


Modo de polarización de un LED



Disposición de los LEDs en un display de 7 segmentos

Representación de los dígitos del 0 al 9 en un display de 7 segmentos

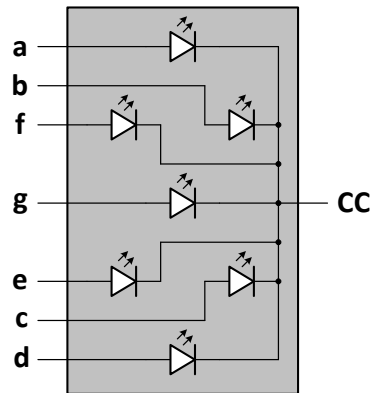


Tema 4. Bloques funcionales combinacionales

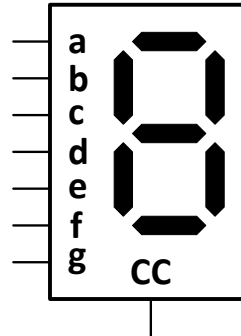
Decodificadores (Conversores de BCD a 7 segmentos)

Existen dos tipos de visualizadores de 7 segmentos: de ánodo común y de cátodo común.

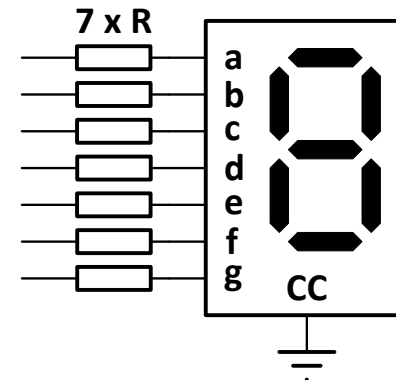
Display de siete segmentos de cátodo común



Conexión interna de los LEDs

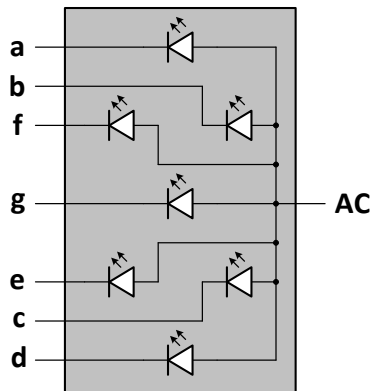


Símbolo

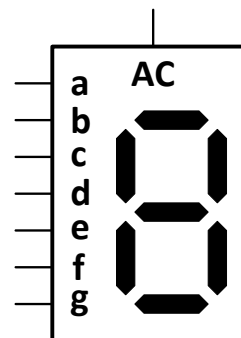


Modo de conexión

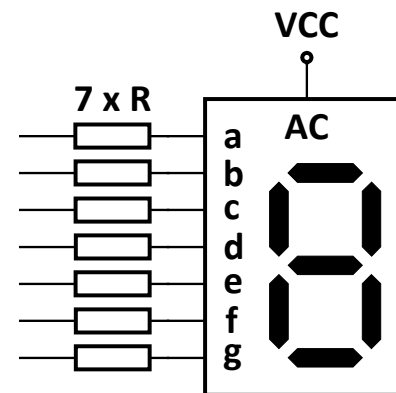
Display de siete segmentos de ánodo común



Conexión interna de los LEDs



Símbolo



Modo de conexión

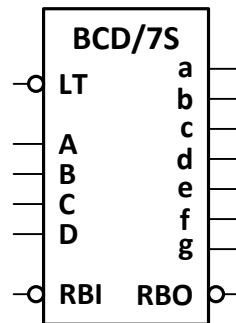
Tema 4. Bloques funcionales combinacionales

Decodificadores (Conversores de BCD a 7 segmentos)

Tipos de conversores de BCD a 7 segmentos

Dado que existen dos configuraciones diferentes de displays de siete segmentos (cátodo común y ánodo común), también existen dos tipos de conversores de BCD a 7 segmentos, para controlar a ambos tipos de visualizadores.

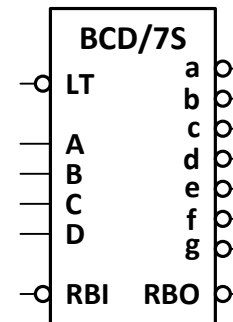
Conversores BCD-7 segmentos para displays de cátodo común



Salidas activas
a nivel alto

LT	RBI	D	C	B	A	a	b	c	d	e	f	g	RBO
0	X	X	X	X	X	1	1	1	1	1	1	1	1
1	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	0	0	0	0	1	1	1	1	1	1	0	1
1	X	0	0	0	1	0	1	1	0	0	0	0	1
1	X	0	0	1	0	1	1	0	1	1	0	1	1
1	X	0	0	1	1	1	1	1	1	0	0	1	1
1	X	0	1	0	0	0	1	1	0	0	1	1	1
1	X	0	1	0	1	1	0	1	1	0	1	1	1
1	X	0	1	1	0	1	0	1	1	1	1	1	1
1	X	0	1	1	1	1	1	1	0	0	0	0	1
1	X	1	0	0	0	1	1	1	1	1	1	1	1
1	X	1	0	0	1	1	1	1	1	0	1	1	1

Conversores BCD-7 segmentos para displays de ánodo común



Salidas activas
a nivel bajo

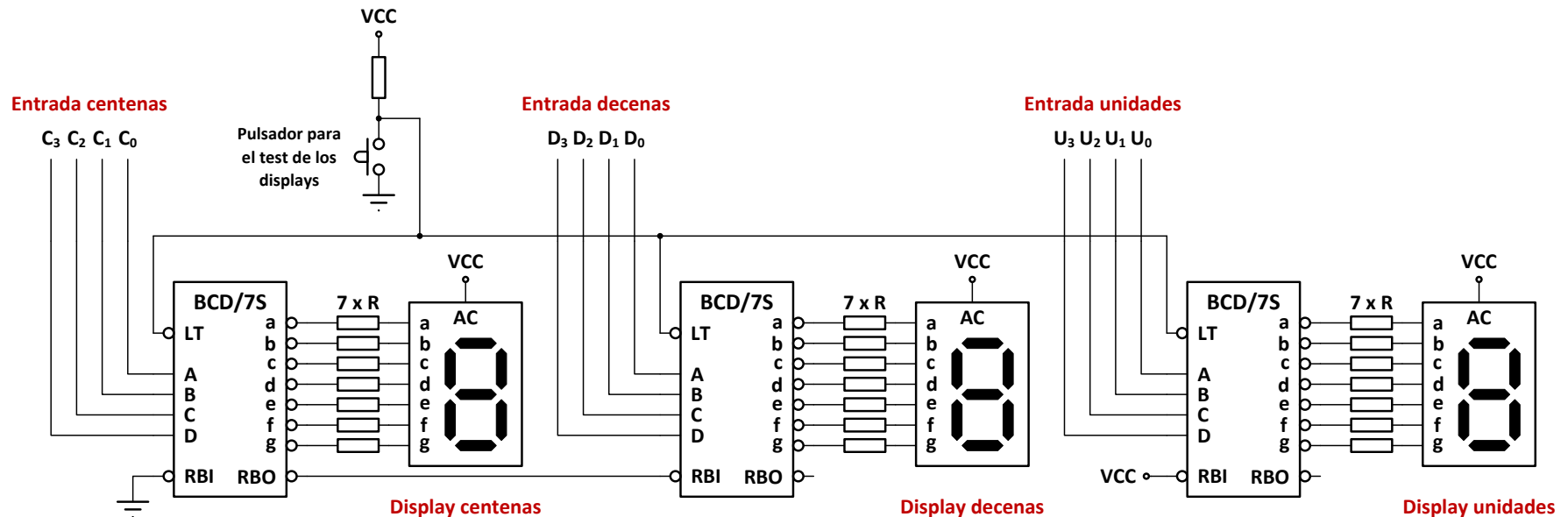
LT	RBI	D	C	B	A	a	b	c	d	e	f	g	RBO
0	X	X	X	X	X	0	0	0	0	0	0	0	1
1	0	0	0	0	0	1	1	1	1	1	1	1	0
1	1	0	0	0	0	0	0	0	0	0	0	1	1
1	X	0	0	0	1	1	0	0	1	1	1	1	1
1	X	0	0	1	0	0	0	1	0	0	1	0	1
1	X	0	0	1	1	0	0	0	1	1	0	0	1
1	X	0	1	0	0	1	0	0	1	1	0	0	1
1	X	0	1	0	1	0	1	0	0	1	0	0	1
1	X	0	1	1	0	0	1	0	0	0	0	0	1
1	X	0	1	1	1	0	0	0	1	1	1	1	1
1	X	1	0	0	0	0	0	0	0	0	0	0	1
1	X	1	0	0	1	0	0	0	0	1	0	0	1

Tema 4. Bloques funcionales combinacionales

Decodificadores (Conversores de BCD a 7 segmentos)

Conexión de varios conversores de BCD/7 segmentos usando las líneas RBI y RBO

En este circuito no aparecerán los ceros a la izquierda correspondientes a las centenas y las decenas. Los ceros en las unidades siempre se visualizarán.



Tema 4. Bloques funcionales combinacionales

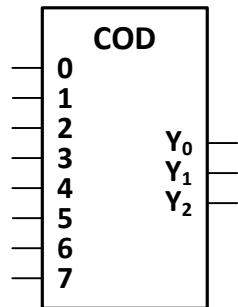
Codificadores

Los codificadores son dispositivos MSI que realizan la operación inversa a la realizada por los decodificadores.

Generalmente, poseen 2^n ($0 - 2^n - 1$) entradas y n salidas.

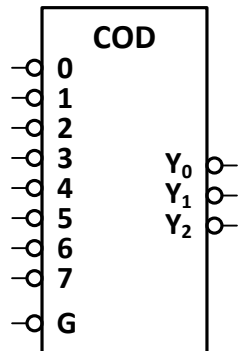
Su funcionamiento es tal, que cuando una de las entradas adopta su estado lógico activo, aparece a la salida la combinación binaria correspondiente al valor decimal asignado a dicha entrada.

Símbolo y tabla de verdad de un codificador de 8 a 3 líneas con entradas y salidas directas



X ₇	X ₆	X ₅	X ₄	X ₃	X ₂	X ₁	X ₀	Y ₂	Y ₁	Y ₀
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

Símbolo y tabla de verdad de un codificador de 8 a 3 líneas con entradas y salidas negadas y entrada de habilitación



G	X ₇	X ₆	X ₅	X ₄	X ₃	X ₂	X ₁	X ₀	Y ₂	Y ₁	Y ₀
1	X	X	X	X	X	X	X	X	1	1	1
0	1	1	1	1	1	1	1	0	1	1	1
0	1	1	1	1	1	1	0	1	1	1	0
0	1	1	1	1	1	0	1	1	1	0	1
0	1	1	1	1	0	1	1	1	1	0	0
0	1	1	1	0	1	1	1	1	0	1	1
0	1	1	0	1	1	1	1	1	0	1	0
0	1	0	1	1	1	1	1	1	0	0	1
0	0	1	1	1	1	1	1	1	0	0	0

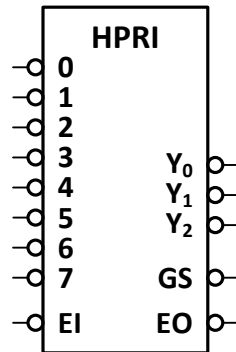
Tema 4. Bloques funcionales combinacionales

Codificadores

Codificadores con prioridad

En los codificadores sin prioridad, como los vistos anteriormente, en cada instante debe estar activa una (y solo una) de las entradas. En caso de activarse varias entradas al mismo tiempo, la salida será igual a la suma lógica de las combinaciones correspondientes a cada una de las entradas activas, y por tanto será errónea.

Para evitar esta limitación de funcionamiento se han desarrollado los **codificadores con prioridad**, en los cuales pueden estar activas varias entradas simultáneamente. Cuando esto ocurre, la combinación de salida será la correspondiente a la entrada activa de mayor valor decimal, si se trata de un **codificador con prioridad alta** (HPRI), o a la de menor valor decimal, en el caso de un **codificador con prioridad baja** (LPRI).



EI	X ₇	X ₆	X ₅	X ₄	X ₃	X ₂	X ₁	X ₀	Y ₂	Y ₁	Y ₀	GS	EO
1	X	X	X	X	X	X	X	X	1	1	1	1	1
0	1	1	1	1	1	1	1	1	1	1	1	1	0
0	1	1	1	1	1	1	1	0	1	1	1	0	1
0	1	1	1	1	1	1	0	X	1	1	0	0	1
0	1	1	1	1	1	0	X	X	1	0	1	0	1
0	1	1	1	1	0	X	X	X	1	0	0	0	1
0	1	1	1	0	X	X	X	X	0	1	1	0	1
0	1	1	0	X	X	X	X	X	0	1	0	0	1
0	1	0	X	X	X	X	X	X	0	0	1	0	1
0	0	X	X	X	X	X	X	X	0	0	0	0	1

Funcionamiento del circuito

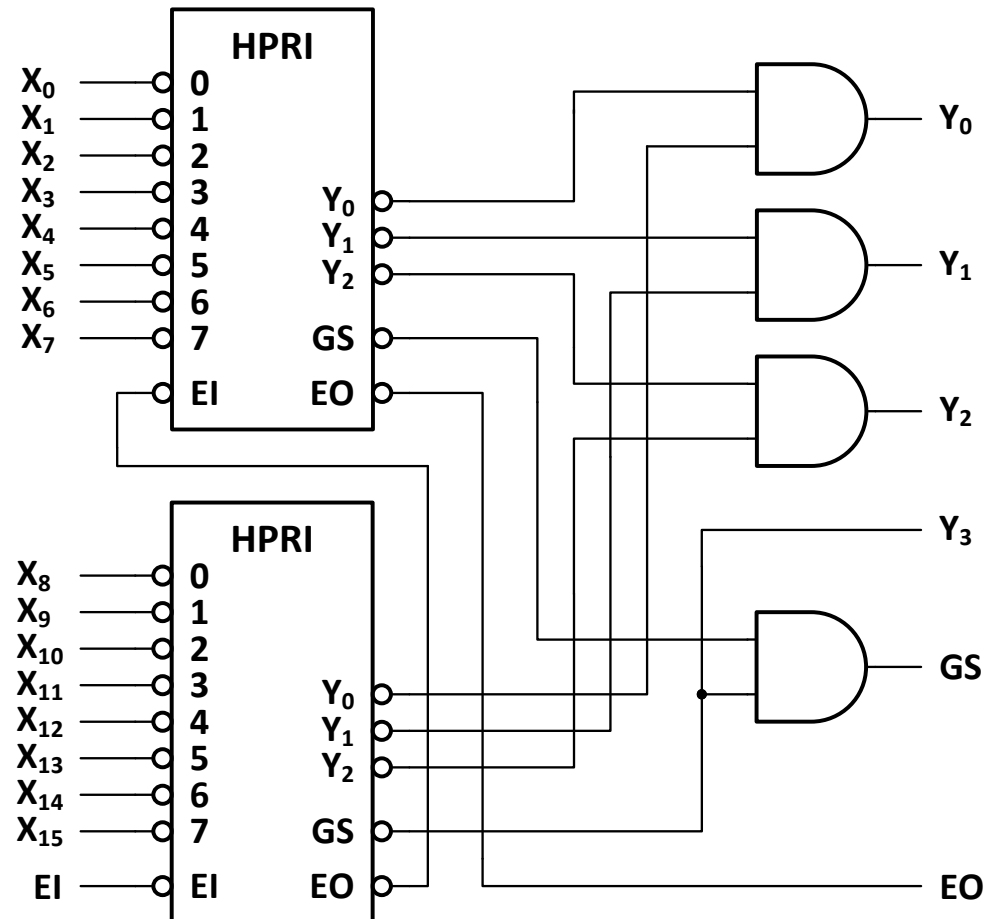
- El circuito sólo codifica cuando se aplica a su entrada de habilitación **EI** (*Enable Input*) un nivel bajo.
- Si el circuito está habilitado y no tiene ninguna entrada activa se activa la salida de habilitación **EO** (*Enable Output*).
- Si el circuito está habilitado y alguna de las entradas está activa se activa la salida **GS** (*Group Signal*) y la combinación de salida **Y₂Y₁Y₀** indica cual es la entrada activa con mayor valor decimal (en binario y de forma complementada).

Tema 4. Bloques funcionales combinacionales

Codificadores (Asociación de codificadores)

Cuando se necesita establecer la prioridad entre un número de líneas superior al número de entradas de un circuito, es preciso realizar una asociación de codificadores.

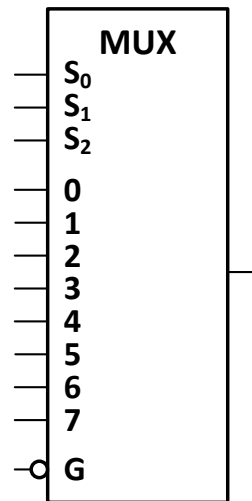
El siguiente diagrama muestra el modo de obtener un codificador con prioridad alta de 16 a 4 líneas usando codificadores de prioridad alta de 8 a 3 líneas.



Tema 4. Bloques funcionales combinacionales

Multiplexores

Un **multiplexor** es un circuito combinacional que posee **n** líneas de selección, **m** canales de entrada (numerados de 0 a m-1) y una salida, tal que $m = 2^n$. Su funcionamiento es tal que la combinación binaria aplicada a las líneas de selección determina cual es el canal de entrada cuya información aparece en la salida del circuito.



G	S ₂	S ₁	S ₀	D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀	Y
1	X	X	X	X	X	X	X	X	X	X	X	0
0	0	0	0	X	X	X	X	X	X	X	A	A
0	0	0	1	X	X	X	X	X	X	B	X	B
0	0	1	0	X	X	X	X	X	C	X	X	C
0	0	1	1	X	X	X	X	D	X	X	X	D
0	1	0	0	X	X	X	E	X	X	X	X	E
0	1	0	1	X	X	F	X	X	X	X	X	F
0	1	1	0	X	H	X	X	X	X	X	X	H
0	1	1	1	I	X	X	X	X	X	X	X	I

Existen en el mercado multiplexores de diferentes tamaños en circuito integrado. Los tamaños más usuales de multiplexores en un mismo encapsulado son:

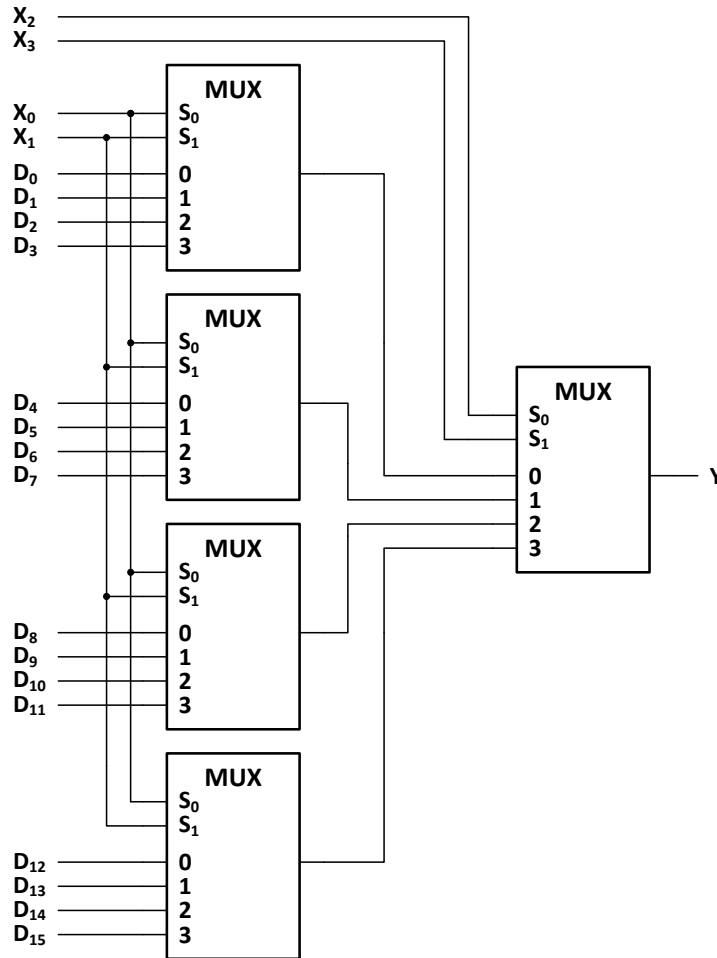
- **Multiplexor de 8 canales (74151).**
- **Multiplexor de 16 canales con salida negada (74150).**
- **Doble multiplexor de 4 canales con líneas de selección comunes (74153).**
- **Cuádruple multiplexor de dos canales con líneas de habilitación y de selección comunes (74157).**

La principal aplicación de los multiplexores es el envío a un único destino de la información procedente de varias fuentes. Los datos a seleccionar pueden ser de un solo bit o combinaciones de varios bits (2, 4, 8, etc.).

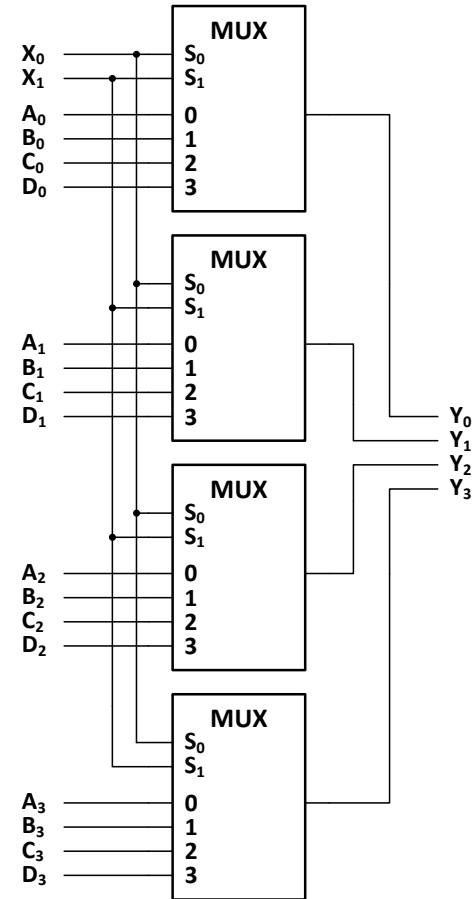
Tema 4. Bloques funcionales combinacionales

Multiplexores (Asociación de multiplexores)

Existen dos razones por las que puede ser necesario asociar multiplexores: aumentar el número de canales o generar buses de información.



Aumento del número de canales



Generación de un bus de información

Tema 4. Bloques funcionales combinacionales

Multiplexores (Realización de funciones lógicas con multiplexores)

Otra aplicación de los multiplexores es la generación de funciones lógicas. Para implementar una función de n variables se necesita un multiplexor con sólo $n-1$ líneas de selección.

Para generar funciones con multiplexores se pueden emplear dos métodos:

- Método algebraico.
- Método tabular.

Seguidamente se estudia el modo de aplicar el segundo de ellos.

Método tabular para la generación de funciones mediante multiplexores

- Se parte de una de las expresiones canónicas numéricas de la función a generar.
- Se representa un mapa con dos filas, asignando a las columnas las diferentes combinaciones de las variables de menor peso de la función, y a las filas los dos valores (0 y 1) de la variable de mayor peso.
- Las variables de las columnas se ordenan de mayor a menor peso y se codifican empleando el código binario natural.
- Cada columna representa una entrada de datos del multiplexor. Así pues, las variables asignadas a las columnas serán las que se conecten a las líneas de selección del multiplexor.
- Se representa dentro de cada casilla de la tabla el valor que posee la función para la combinación asociada a la misma.
- Se evalúa el contenido de cada columna, para determinar qué valor hay que aplicar al canal correspondiente del multiplexor, representándolo en la parte inferior de la misma:
 - Si una columna posee dos "0", o un "0" y una "X" se colocará un "0".
 - Si una columna posee dos "1", o un "1" y una "X" se colocará un "1".
 - Si una columna posee un "0", en la primera fila y un "1" en la segunda se colocará la variable de mayor peso directa.
 - Si una columna posee un "1", en la primera fila y un "0" en la segunda se colocará la variable de mayor peso negada.
 - Si una columna posee dos "X", se puede colocar "0" o "1" indistintamente.
- Finalmente se representa el símbolo del multiplexor, conectando a las líneas de selección las variables de menor peso de la función (teniendo en cuenta los pesos) y a los diferentes canales los valores indicados en la tabla debajo de cada columna.

Tema 4. Bloques funcionales combinacionales

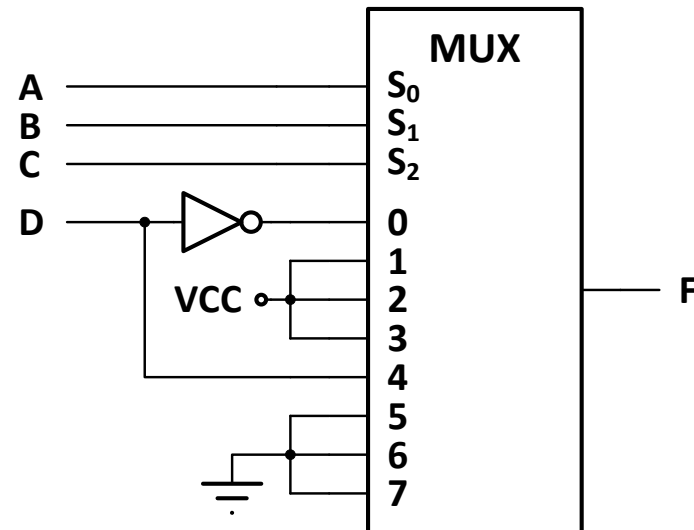
Multiplexores (Realización de funciones lógicas con multiplexores)

Ejemplo: Realizar la función $F(D, C, B, A) = \sum_4 (0, 1, 2, 3, 9, 11, 12) + \sum_\phi (6, 7, 10, 15)$ mediante un multiplexor del tamaño más adecuado.

Al tratarse de una función de 4 variables se necesitará un multiplexor de 8 canales (es decir, con tres líneas de selección).

	D	C	B	A	F
0	0	0	0	0	1
1	0	0	0	1	1
2	0	0	1	0	1
3	0	0	1	1	1
4	0	1	0	0	0
5	0	1	0	1	0
6	0	1	1	0	X
7	0	1	1	1	X
8	1	0	0	0	0
9	1	0	0	1	1
10	1	0	1	0	X
11	1	0	1	1	1
12	1	1	0	0	1
13	1	1	0	1	0
14	1	1	1	0	0
15	1	1	1	1	X

CBA		000	001	010	011	100	101	110	111
D	0	1 ₀	1 ₁	1 ₂	1 ₃	0 ₄	0 ₅	X ₆	X ₇
	1	0 ₈	1 ₉	X ₁₀	1 ₁₁	1 ₁₂	0 ₁₃	0 ₁₄	X ₁₅
		$\bar{D}$	1	1	1	D	0	0	X



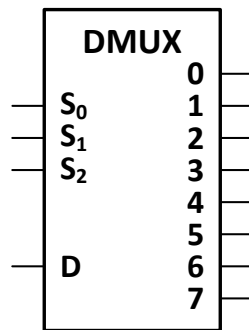
Tema 4. Bloques funcionales combinacionales

Demultiplexores

Un **demultiplexor** es un sistema combinacional con una entrada de información, **n** líneas de selección y **m** salidas ($m = 2^n$) numeradas de 0 a $m-1$.

Su funcionamiento es tal que cuando se aplica una combinación a las líneas de selección, la información presente en la entrada de datos aparece en la salida cuyo valor decimal corresponde a dicha combinación. El resto de las salidas mantienen su valor por defecto, que puede ser "0" o "1".

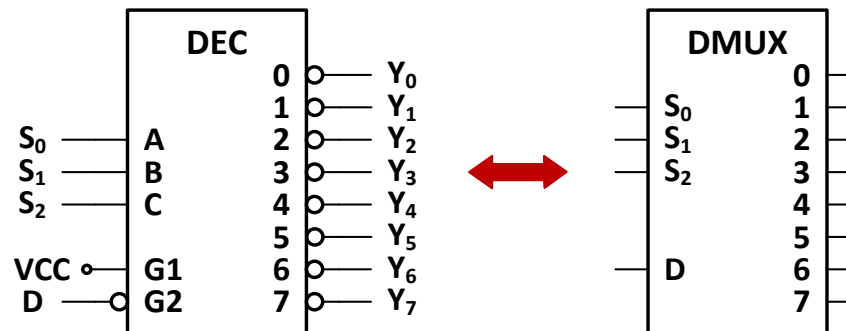
Símbolo y tabla de verdad de un demultiplexor de 1 a 8 líneas



S ₂	S ₁	S ₀	D	Y ₀	Y ₁	Y ₂	Y ₃	Y ₄	Y ₅	Y ₆	Y ₇
0	0	0	E	E	0	0	0	0	0	0	0
0	0	1	E	0	E	0	0	0	0	0	0
0	1	0	E	0	0	E	0	0	0	0	0
0	1	1	E	0	0	0	E	0	0	0	0
1	0	0	E	0	0	0	0	E	0	0	0
1	0	1	E	0	0	0	0	0	E	0	0
1	1	0	E	0	0	0	0	0	0	E	0
1	1	1	E	0	0	0	0	0	0	0	E

Utilización de un decodificador como demultiplexor

En realidad no existen demultiplexores integrados como tal, sino que se utilizan decodificadores como demultiplexores.



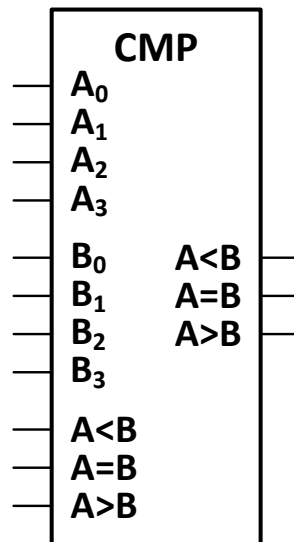
Tema 4. Bloques funcionales combinacionales

Comparadores

Un **comparador** es un circuito que indica si dos combinaciones expresadas en binario natural son iguales, y si no es así cual de ellas es mayor.

Existen comparadores para combinaciones de 4 bits y de 8 bits.

Comparador de 4 bits



ENTRADAS				SALIDAS		
A - B	A < B	A = B	A > B	A < B	A = B	A > B
A < B	X	X	X	1	0	0
A > B	X	X	X	0	0	1
A = B	1	0	0	1	0	0
A = B	0	1	0	0	1	0
A = B	0	0	1	0	0	1

- Si las combinaciones de entrada A y B son diferentes, se activarán las salidas A < B o A > B, según corresponda.
- Si las combinaciones de entrada son iguales, las salidas A < B, A = B y A > B tomarán los mismos valores que las entradas del mismo nombre.

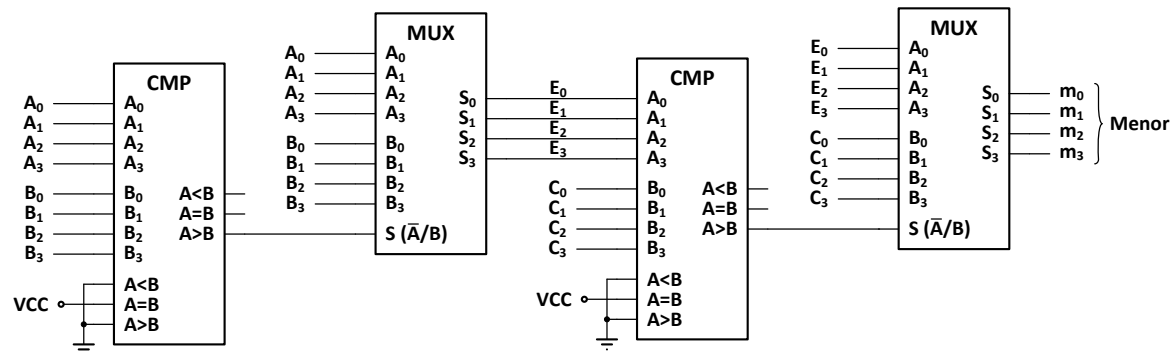
Tema 4. Bloques funcionales combinacionales

Comparadores

Una de las aplicaciones de los comparadores es la de seleccionar combinaciones de forma automática en función de su valor en combinación con multiplexores.

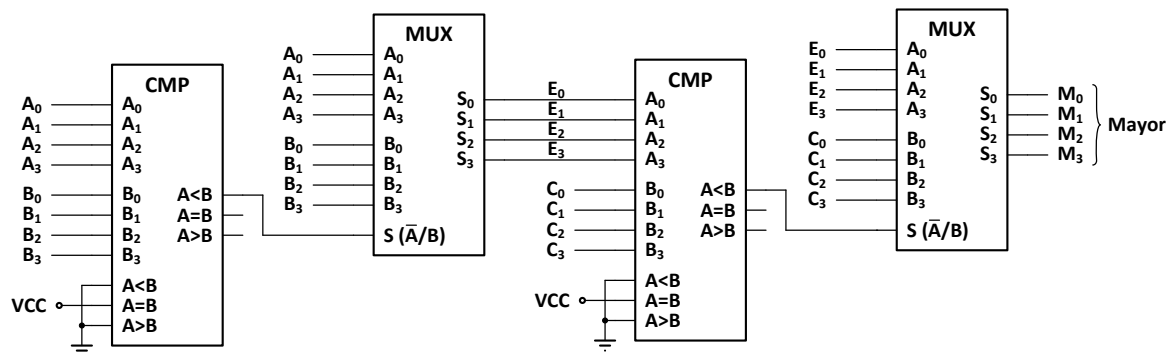
Selección de la combinación de menor valor

El circuito de la figura proporciona a su salida la combinación de menor valor entre las combinaciones de entrada **A**, **B** y **C**. Para ello se emplean dos comparadores de cuatro bits y dos multiplexores cuádruples de dos canales.



Selección de la combinación de mayor valor

El circuito de la figura proporciona a su salida la combinación de mayor valor entre las combinaciones de entrada **A**, **B** y **C**. Para ello se emplean dos comparadores de cuatro bits y dos multiplexores cuádruples de dos canales.

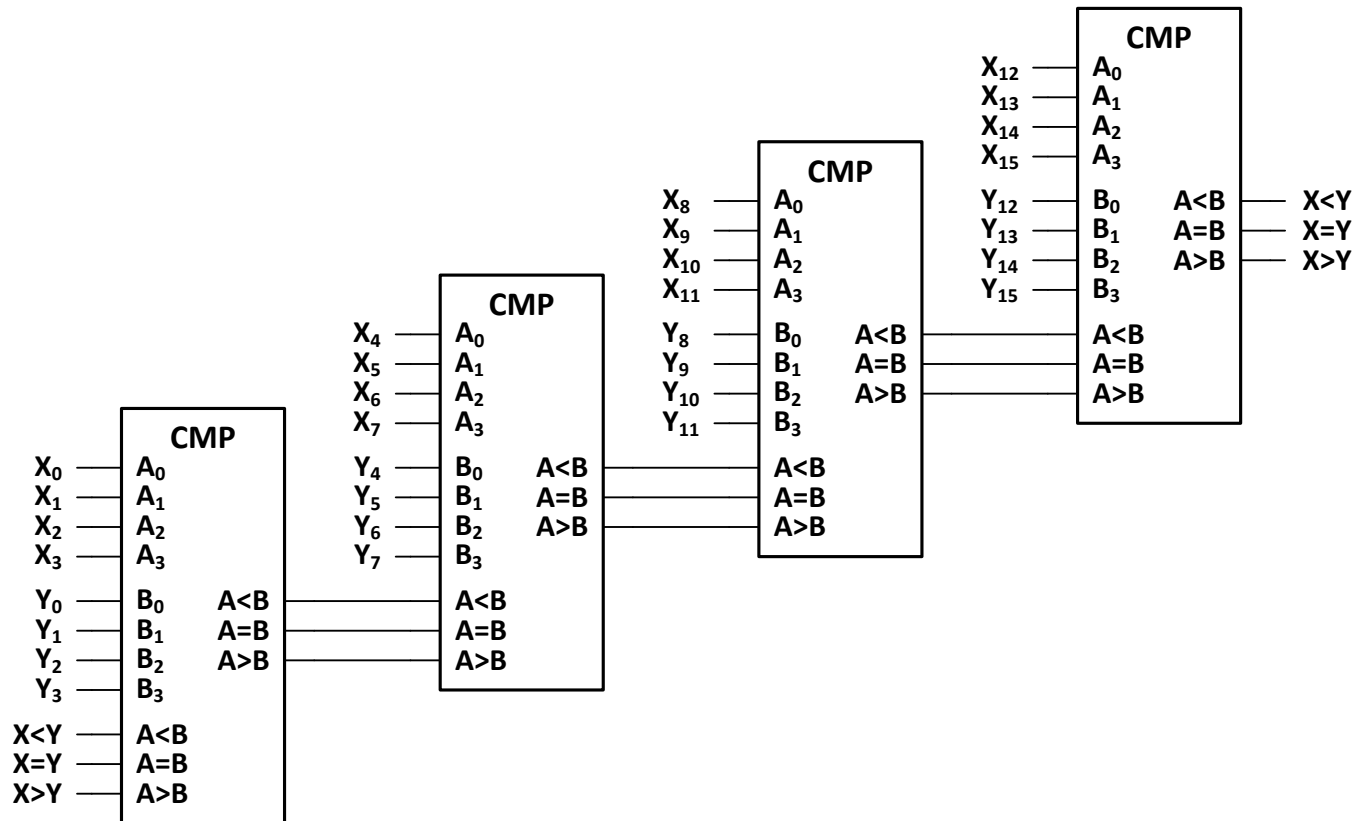


Tema 4. Bloques funcionales combinacionales

Comparadores (Asociación de comparadores)

Los comparadores disponibles en el mercado suelen tener un tamaño de 4 bits, como el estudiado anteriormente. No obstante, si se necesita comparar combinaciones de una longitud superior, se puede obtener el comparador necesario mediante la conexión en cascada de varios de estos circuitos.

Para ello, las salidas de cada comparador se conectan a las entradas del mismo nombre del comparador que recibe los cuatro bits de peso inmediatamente superior de las combinaciones de entrada. Las entradas $A < B$, $A = B$ y $A > B$ del comparador resultante serán las correspondientes al comparador al que se aplican los cuatro bits de menos peso de ambas combinaciones, y las salidas del mismo serán las de aquel al que se aplican los 4 bits de más peso.



Tema 4. Bloques funcionales combinacionales

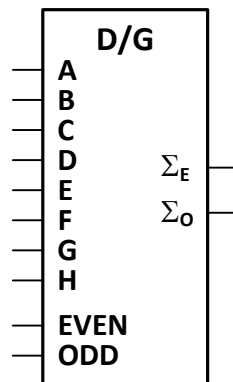
Detectores / generadores de paridad

En los sistemas digitales, sobre todo en aquellos capaces de detectar o corregir errores, puede ser necesario detectar la paridad de combinaciones binarias o generar bits de paridad a partir de las mismas.

La paridad de una combinación binaria viene determinada por el número de unos que ésta posee. Una combinación tiene paridad impar (**ODD**) cuando contiene un número impar de unos, y paridad par (**EVEN**) en caso contrario.

- **Detectar** la paridad de una combinación es averiguar si el número de unos que posee es par o impar. Cuando se **detecta paridad impar** sobre una combinación, el resultado será 1 si el número de unos de la misma es impar y 0 en caso contrario. Por el contrario, cuando se **detecta paridad par**, el resultado será 1 si el número de unos de la misma es par (incluyendo el 0) y 0 en caso contrario. Por tanto, el resultado de detectar paridad impar es el contrario de detectar paridad par, y viceversa.
- **Generar** paridad es obtener un nuevo bit a partir de una combinación dada de n bits, cuyo valor sea tal que al ser añadido a la misma, haga que la combinación de n+1 bits resultante posea una determinada paridad (par o impar). Cuando se **genera paridad impar** el número de unos de la combinación resultante debe ser impar, por lo que si la combinación de partida tiene paridad par el valor del bit generado debe ser 1 y si tiene paridad impar dicho valor debe ser 0. Por el contrario, cuando se **genera paridad par** el número de unos de la combinación resultante debe ser par, por lo que si la combinación de partida tiene paridad par el valor del bit generado debe ser 0 y si tiene paridad impar dicho valor debe ser 1. Por tanto, el resultado de generar paridad impar es el contrario de generar paridad par, y viceversa.

Para detectar y generar paridad en los sistemas digitales se suele hacer uso de unos circuitos combinacionales denominados **detectores/generadores de paridad**. Estos circuitos permiten, según se configuren, la detección o generación tanto de paridad par como impar.



PARIDAD COMBINACIÓN ENTRADA	EVEN	ODD	Σ_E	Σ_O	MODO
PAR	1	0	1	0	DETECTOR
IMPAR	1	0	0	1	
PAR	0	1	0	1	GENERADOR
IMPAR	0	1	1	0	
X	0	0	1	1	NO FUNCIONA
X	1	1	0	0	

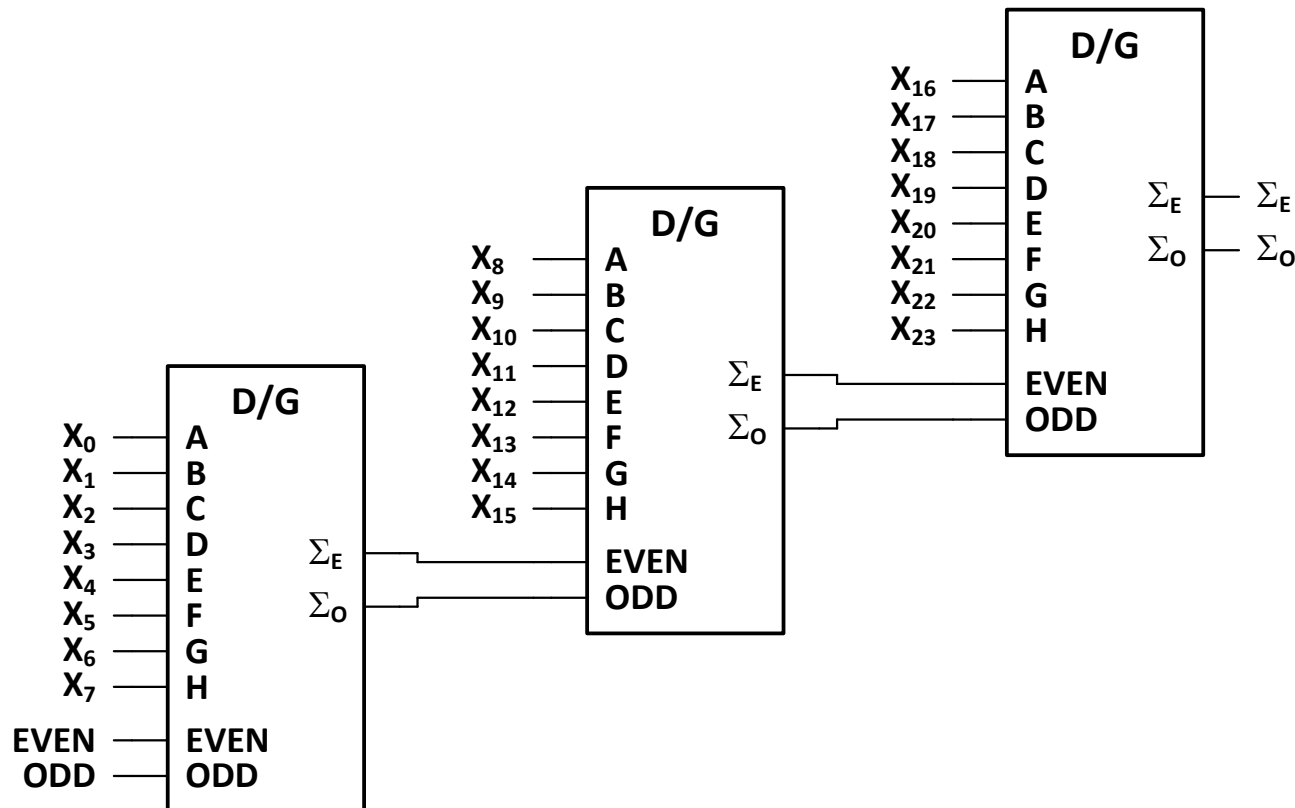
Tema 4. Bloques funcionales combinacionales

Detectores / generadores de paridad (Asociación de detectores / generadores de paridad)

Además de servir para la configuración del circuito como detector o como generador de paridad, las entradas **EVEN** y **ODD** también permiten obtener un circuito con más entradas de datos mediante la asociación de varios bloques.

Para ello, las salidas par (Σ_E) e impar (Σ_O) de cada circuito deben conectarse, respectivamente, a las entradas par (**EVEN**) e impar (**ODD**) del siguiente.

Ejemplo: Obtener un detector / generador de paridad para combinaciones de 24 bits.



Tema 4. Bloques funcionales combinacionales

Lenguaje VHDL (Nociones básicas)

En esta asignatura se usará el lenguaje **VHDL** para la descripción textual de sistemas digitales. No obstante, este lenguaje no solo permite la descripción de circuitos digitales, sino que permite el modelado de sistemas de cualquier tipo.

Las características básicas del lenguaje VHDL son las siguientes :

- En VHDL no se hace distinción entre mayúsculas y minúsculas.
- Cualquier texto existente en una línea de código después de dos guiones seguidos (--) es considerado un comentario.
- Cada módulo descriptivo y cada asignación de valor a una señal debe terminar con un punto y coma (;).
- Los elementos básicos usados en la descripción digital son las señales (**signal**).
- VHDL es un lenguaje fuertemente tipado, es decir, el tipo de cada señal debe definirse previamente al uso de la misma, no permitiéndose asignaciones a tipos de datos incorrectos.
- Para la declaración de señales binarias suele emplearse el tipo **std_logic** (abreviatura de *standard logic* ⇒ lógica estándar). En este tipo, los valores de las señales se expresan siempre entre comillas. Existen nueve valores posibles:
 - '0', '1': Valores booleanos típicos.
 - 'X': Desconocido.
 - 'Z': Alta impedancia.
 - 'U': Sin inicializar (por ejemplo un biestable en su situación original).
 - '-': Condición de no importa (*don't care*).
 - 'L': '0' débil.
 - 'H': '1' débil.
 - 'W': Desconocido débil.

Los 6 primeros valores (los representados en color azul) son los más usados, y con ellos puede describirse cualquiera de los circuitos que se modelarán en la asignatura.

Tema 4. Bloques funcionales combinacionales

Lenguaje VHDL (Nociones básicas)

Un conjunto de señales (bits) constituye un **vector**. Los vectores pueden declararse en forma ascendente o descendente:

- **Ascendente:** Por ejemplo `std_logic_vector (0 to 7)`.
- **Descendente:** Por ejemplo `std_logic_vector (7 downto 0)`. Esta forma se utiliza mucho porque el bit más significativo se corresponde con el de mayor índice.

Los tipos **standard logic** se introdujeron en la normalización realizada por IEEE. Para poder usarlos, hay que incluir en la cabecera de los ficheros fuente la declaración de la librería (**library**) y de los paquetes de la librería que serán usados (**use**), los cuales definirán tales tipos, así como sus operaciones. A continuación se muestra el modo de declarar la librería **ieee** y de activar todos los elementos del paquete **std_logic_1164** incluido en la misma.

```
library ieee;  
  
use ieee.std_logic_1164.all;
```

- Si se activa el paquete **ieee.std_logic_unsigned** las operaciones se realizarán en binario natural. Por el contrario, si se desea trabajar con números negativos expresados según el convenio del complemento a 2, se debe usar el paquete **ieee.std_logic_signed**.
- Los paquetes **ieee.std_logic_unsigned** e **ieee.std_logic_signed** incluyen, además, la función **conv_integer (A)**, la cual convierte a entero el valor binario del vector **A**.
- Por otro lado, el paquete **ieee.std_logic_arith** incluye la función **conv_std_logic_vector (B, N)**, que genera una combinación binaria de **N** bits a partir del valor entero **B**, la cual podrá ser almacenada en un vector.

Tema 4. Bloques funcionales combinacionales

Lenguaje VHDL (Operaciones básicas)

Los operadores básicos de VHDL para realizar operaciones entre señales son los siguientes:

Operador de asignación	<code><=</code>
Operadores booleanos	<code>not</code> , <code>and</code> , <code>or</code> , <code>nand</code> , <code>nor</code> , <code>xor</code> , <code>xnor</code>
Operadores de comparación	<code>=</code> , <code>/=</code> , <code>></code> , <code><</code> , <code>>=</code> , <code><=</code>
Operadores aritméticos	<code>+</code> , <code>-</code> , <code>*</code>
Operador de concatenación	<code>&</code> <i>(Permite colocar juntos señales o vectores, formando un vector más largo)</i>

La asignación de valores a las señales puede hacerse directamente o por medio de operaciones entre señales.

Ejemplos:

```
signal A, B, F : std_logic_vector (3 downto 0);
signal M : integer range 0 to 15;
...
A <= "1100";
A <= (3 => '1', 0 => '1', others => '0');
M <= 7;
F <= (not A and B) or (A and not B); -- Equivale a una operación XOR.
F <= A + B; -- Suma aritmética.
```

Tema 4. Bloques funcionales combinacionales

Lenguaje VHDL (Estructura de una descripción en VHDL)

En VHDL, las descripciones poseen tres partes bien diferenciadas:

- **Declaración de librerías y paquetes:** Aquellas que serán usadas en el diseño.

```
library ieee;  
use ieee.std_logic_1164.all;  
use ieee. std_logic_unsigned.all;
```

- **Entidad (módulo de declaración de terminales):** Es la parte del módulo VHDL donde se declaran los terminales del circuito a diseñar, es decir, las entradas, salidas y, en algunos casos, las líneas bidireccionales de E/S. Los puertos (**port**) declarados en la entidad pueden ser de entrada (**in**), de salida (**out**), bidireccionales (**inout**) y adaptados (**buffer**), estos últimos menos usados que los anteriores.

```
entity NOMBRE_DE_LA_ENTIDAD is  
    generic (Declaración de parámetros);  
    port (Declaración de entradas y salidas);  
end NOMBRE_DE_LA_ENTIDAD;
```

- **Arquitectura:** En esta parte es donde se describe el funcionamiento del circuito. Para ello, se pueden emplear varios tipos de descripción, que se estudiarán posteriormente.

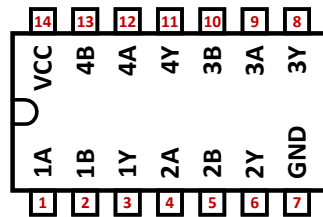
```
architecture NOMBRE_DE_LA_ARQUITECTURA of NOMBRE_DE_LA_ENTIDAD is  
    component (Opcional) ⇒ Declaración de componentes.  
    signal (Opcional) ⇒ Declaración de señales internas.  
    constant (Opcional) ⇒ Declaración de constantes.  
begin  
    Descripción del funcionamiento del sistema (instrucciones concurrentes y secuenciales).  
end NOMBRE_DE_LA_ARQUITECTURA;
```

Tema 4. Bloques funcionales combinacionales

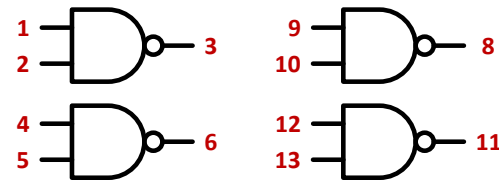
Lenguaje VHDL (Ejemplo de descripción sencilla en VHDL)

Consideremos un circuito integrado sencillo como el **7400**, que contiene cuatro puertas **NAND** de dos entradas.

La **Entidad** se podría representar gráficamente mediante el encapsulado del chip con el patillaje, mientras que la **arquitectura** se correspondería con la función realizada por el circuito, que vendría determinada por el tipo de puertas que posee.



Entidad



Arquitectura

A continuación se muestran dos formas de modelar en VHDL este mismo circuito.

```
library ieee;
use ieee.std_logic_1164.all;

entity CUATRO_NAND is
    port ( A1, B1, A2, B2, A3, B3, A4, B4: in std_logic;
          Y1, Y2, Y3, Y4: out std_logic);
end CUATRO_NAND;

architecture A1_CUATRO_NAND of CUATRO_NAND is
begin
    Y1 <= A1 nand B1;
    Y2 <= A2 nand B2;
    Y3 <= A3 nand B3;
    Y4 <= A4 nand B4;
end A1_CUATRO_NAND;
```

```
library ieee;
use ieee.std_logic_1164.all;

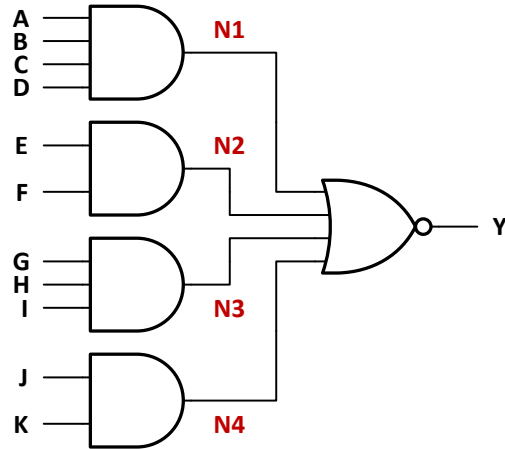
entity CUATRO_NAND is
    port ( A, B: in std_logic_vector (1 to 4);
          Y: out std_logic_vector (1 to 4));
end CUATRO_NAND ;

architecture A2_CUATRO_NAND of CUATRO_NAND is
begin
    Y <= A nand B;
end A2_CUATRO_NAND;
```

Tema 4. Bloques funcionales combinacionales

Lenguaje VHDL (Ejercicio)

Modele en VHDL el circuito **7464**, cuyos diagrama lógico y expresión algebraica se muestran a continuación.



$$Y = \overline{ABCD + EF + GHI + JK}$$

Se pueden emplear señales internas (**signal**), que se declaran dentro de la arquitectura antes del **begin**, para definir los puntos de entrada a la puerta NOR. De esta forma resulta más fácil de modelar y de interpretar la función completa.

En este ejemplo se han empleado cuatro señales internas (**N1**, **N2**, **N3** y **N4**), que se corresponden con las salidas de las puertas AND, para facilitar la descripción del circuito.

```
library ieee;
use ieee.std_logic_1164.all;

entity CIRCUITO_AND_NOR is
    port ( A, B, C, D, E, F, G, H, I, J, K: in std_logic;
          Y: out std_logic);
end CIRCUITO_AND_NOR;

architecture A_CIRCUITO_AND_NOR of CIRCUITO_AND_NOR is
    signal N1, N2, N3, N4: std_logic;
begin
    N1 <= A and B and C and D;
    N2 <= E and F;
    N3 <= G and H and I;
    N4 <= J and K;
    Y <= not (N1 or N2 or N3 or N4);
end A_CIRCUITO_AND_NOR;
```

Tema 4. Bloques funcionales combinacionales

Lenguaje VHDL (Instrucciones concurrentes)

Las **instrucciones concurrentes** en VHDL son aquellas que se ejecutan directamente sobre las señales.

Sobre una determinada señal sólo se puede aplicar una instrucción concurrente. En caso de aplicar varias instrucciones concurrentes sobre una misma señal se produciría un error al intentar asignar a ésta varios valores diferentes.

Las instrucciones concurrentes pueden ser de tres tipos:

- **Fijas:** Asignan a la señal un valor fijo, o bien, el resultado de una operación entre señales, entre valores, o entre ambas cosas.

SEÑAL <= **Valor**;

SEÑAL <= **Operación**;

- **Condicionales:** Asignan a la señal diferentes valores en función de si se cumplen unas condiciones u otras. La primera condición indicada es la que posee más prioridad y la última la que menos. Si no se cumple ninguna de las condiciones expresadas en la sentencia, se asigna a la señal el valor por defecto indicado tras el último **else**.

SEÑAL <= **Valor_1 when Condicion_1 else**

Valor_2 when Condicion_2 else

...

Valor_n when Condicion_n else

Valor_por_defecto;

- **Múltiples:** Asignan a la señal diferentes valores en función del valor que posea una segunda señal (**SEÑAL DE REFERENCIA**). Si la señal de referencia no posee ninguno de los valores enumerados en la sentencia, se asigna a la señal el valor por defecto indicado antes de **when others**;

with SEÑAL_DE_REFERENCIA select

SEÑAL <= **Valor_1 when Valor_señal_de_referencia_1,**

Valor_2 when Valor_señal_de_referencia_2,

...

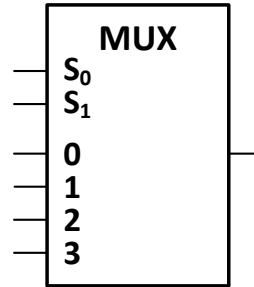
Valor_n when Valor_señal_de_referencia_n,

Valor_por_defecto when others;

Tema 4. Bloques funcionales combinacionales

Lenguaje VHDL (Ejemplos de uso de instrucciones concurrentes)

Ejemplo 1: Describir el multiplexor de 4 canales de la figura usando **instrucciones concurrentes** condicionales y múltiples.



Usando instrucciones concurrentes condicionales

```
library ieee;
use ieee.std_logic_1164.all;

entity MUX_4 is
    port ( C0, C1, C2, C3, S1, S0: in std_logic;
          Y : out std_logic);
end MUX_4 ;

architecture ICC_MUX_4 of MUX_4 is
    signal SELECCION: std_logic_vector (1 downto 0);
begin
    SELECCION <= S1 & S0;
    Y <=  C0 when SELECCION = "00" else
         C1 when SELECCION = "01" else
         C2 when SELECCION = "10" else
         C3;
end ICC_MUX_4;
```

Usando instrucciones concurrentes múltiples

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;

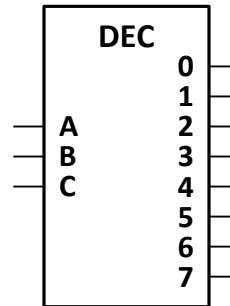
entity MUX_4 is
    port ( C0, C1, C2, C3: in std_logic;
          S: in std_logic_vector (1 downto 0);
          Y : out std_logic);
end MUX_4 ;

architecture ICM_MUX_4 of MUX_4 is
    signal SELECCION: integer range 0 to 3;
begin
    SELECCION <= conv_integer (S);
    with SELECCION select
        Y <= C0 when 0,
            C1 when 1,
            C2 when 2,
            C3 when others;
end ICM_MUX_4 ;
```

Tema 4. Bloques funcionales combinacionales

Lenguaje VHDL (Ejemplos de uso de instrucciones concurrentes)

Ejemplo 2: Describir el decodificador de 3 a 8 líneas de la figura usando **instrucciones concurrentes** condicionales y múltiples.



Usando instrucciones concurrentes condicionales

```
library ieee;
use ieee.std_logic_1164.all;

entity DEC_8 is
    port (A, B, C: in std_logic;
          SALIDAS: out std_logic_vector (0 to 7));
end DEC_8;

architecture ICC_DEC_8 of DEC_8 is
    signal SELECCION: std_logic_vector (2 downto 0);
begin
    SELECCION <= C & B & A;
    SALIDAS <= "10000000" when SELECCION = "000" else
               "01000000" when SELECCION = "001" else
               "00100000" when SELECCION = "010" else
               "00010000" when SELECCION = "011" else
               "00001000" when SELECCION = "100" else
               "00000100" when SELECCION = "101" else
               "00000010" when SELECCION = "110" else
               "00000001";
end ICC_DEC_8;
```

Usando instrucciones concurrentes múltiples

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;

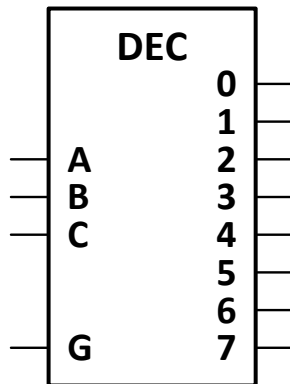
entity DEC_8 is
    port (A, B, C: in std_logic;
          SALIDAS: out std_logic_vector (0 to 7));
end DEC_8;

architecture ICM_DEC_8 of DEC_8 is
    signal SELECCION: std_logic_vector (2 downto 0);
    signal SELECCION_INT: integer range 0 to 7;
begin
    SELECCION <= C & B & A;
    SELECCION_INT <= conv_integer (SELECCION);
    with SELECCION_INT select
        SALIDAS <= "10000000" when 0,
                   "01000000" when 1,
                   "00100000" when 2,
                   "00010000" when 3,
                   "00001000" when 4,
                   "00000100" when 5,
                   "00000010" when 6,
                   "00000001" when others;
end ICM_DEC_8;
```


Tema 4. Bloques funcionales combinacionales

Lenguaje VHDL (Ejemplos de uso de instrucciones concurrentes)

Ejemplo 3: Modificar el ejemplo anterior para incluir en el decodificador una línea de habilitación (**G**) activa a nivel alto.



```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;

entity DEC_8_G is
    port ( A, B, C, G: in std_logic;
          SALIDAS: out std_logic_vector (0 to 7));
end DEC_8_G;

architecture A_DEC_8_G of DEC_8_G is
    signal SELECCION: std_logic_vector (2 downto 0);
    signal SELECCION_INT: integer range 0 to 7;
    signal SALIDAS_G: std_logic_vector (0 to 7);
begin
    SELECCION <= C & B & A;
    SELECCION_INT <= conv_integer (SELECCION);
    with SELECCION_INT select
        SALIDAS_G <= "10000000" when 0,
                      "01000000" when 1,
                      "00100000" when 2,
                      "00010000" when 3,
                      "00001000" when 4,
                      "00000100" when 5,
                      "00000010" when 6,
                      "00000001" when others;

    -- Si G = 0 todas las salidas están inactivas.
    SALIDAS <= SALIDAS_G when G = '1' else "00000000";
end A_DEC_8_G;
```

Tema 4. Bloques funcionales combinacionales

Lenguaje VHDL (Instrucciones secuenciales)

Las **instrucciones secuenciales** son aquellas que se ejecutan en secuencia, de forma ordenada desde la primera hasta la última, dentro de un módulo especial denominado proceso (**process**).

Al contrario de lo que ocurre con las instrucciones concurrentes, donde resulta irrelevante el orden en que éstas aparecen en el código, **en el caso de las instrucciones secuenciales es importante la ordenación** de las mismas, ya que al ejecutarse éstas en secuencia, el resultado de la ejecución del proceso dependerá de la disposición de las sentencias dentro del módulo.

Los valores generados por las instrucciones secuenciales no se aplican a las señales correspondientes hasta que finaliza la ejecución del proceso, de manera que si en el interior del módulo se realiza más de una asignación a una determinada señal, el único valor que se aplica a la misma es el correspondiente a la última de las asignaciones. En el caso de las asignaciones condicionales, el valor aplicado a la señal es el correspondiente a la última asignación cuyas condiciones se cumplen.

En consecuencia, durante todo el tiempo de ejecución del proceso (hasta el momento en que se producen las asignaciones) las diferentes señales conservan el valor que poseían al iniciarse la ejecución del mismo.

Para la declaración de un proceso se utiliza el siguiente formato:

NOMBRE_DEL_PROCESO (Opcional): **process** (**Lista de sensibilidades**)

begin

Instrucciones secuenciales;

end process;

- La función de la **lista de sensibilidades** es determinar cuando se ejecutará el proceso. En este campo se especifican los nombres de una o varias señales, separados por comas, de tal modo que cada vez que alguna de las señales indicadas experimenta un cambio de valor se inicia una nueva ejecución del proceso. Si no se especifica ninguna señal en la lista de sensibilidades del proceso, éste se estará ejecutando indefinidamente. En este caso, el proceso debe incluir obligatoriamente en su interior una instrucción de espera (**wait**).
- Un proceso completo equivale a una única instrucción concurrente, y por ello, **no está permitido realizar asignaciones a una misma señal en varios procesos diferentes**.

Tema 4. Bloques funcionales combinacionales

Lenguaje VHDL (Instrucciones secuenciales)

Al igual que las instrucciones concurrentes, las instrucciones secuenciales pueden ser de tres tipos: **fijas**, **condicionales** y **múltiples**.

- **Fijas:** Poseen exactamente el mismo formato que las instrucciones concurrentes fijas.

SEÑAL <= **Valor**;

SEÑAL <= **Operación**;

- **Condicionales:** Asignan a las señales diferentes valores en función de si se cumplen unas condiciones u otras. La primera condición indicada es la que posee más prioridad y la última la que menos. Si no se cumple ninguna de las condiciones expresadas en la sentencia, se asigna a las señales los valores por defecto indicados después de **else**.

```
if Condicion_1 then SEÑAL_1 <= __; SEÑAL_2 <= __; ...; SEÑAL_N <= __;
  elsif Condicion_2 then SEÑAL_1 <= __; SEÑAL_2 <= __; ...; SEÑAL_N <= __;
  ...
  else SEÑAL_1 <= __; SEÑAL_2 <= __; ...; SEÑAL_N <= __;
end if;
```

- **Múltiples:** Asignan a las señales diferentes valores en función del valor que posea una **SEÑAL DE REFERENCIA**. Si la señal de referencia no posee ninguno de los valores enumerados en la sentencia, se asigna a las señales los valores por defecto indicados después de **when others**;

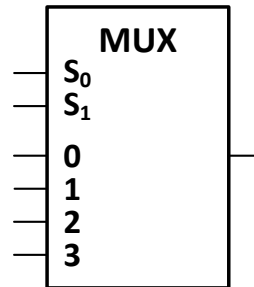
```
case SEÑAL_DE_REFERENCIA is
  when Valor_señal_de_referencia_1 => SEÑAL_1 <= __; SEÑAL_2 <= __; ...; SEÑAL_N <= __;
  when Valor_señal_de_referencia_2 => SEÑAL_1 <= __; SEÑAL_2 <= __; ...; SEÑAL_N <= __;
  ...
  when others => SEÑAL_1 <= __; SEÑAL_2 <= __; ...; SEÑAL_N <= __;
end case;
```

IMPORTANTE: Dentro de un proceso no pueden utilizarse **instrucciones concurrentes** (**when ... else, with ... select**), del mismo modo que las **instrucciones secuenciales** (**if ... else, case ... is**) no pueden ser utilizadas fuera de los procesos.

Tema 4. Bloques funcionales combinacionales

Lenguaje VHDL (Ejemplos de uso de instrucciones secuenciales)

Ejemplo: Describir el multiplexor de 4 canales de la figura usando **instrucciones secuenciales** condicionales y múltiples.



Usando instrucciones secuenciales condicionales

```
library ieee;
use ieee.std_logic_1164.all;

entity MUX_4 is
    port ( C0, C1, C2, C3, S1, S0: in std_logic;
           Y : out std_logic);
end MUX_4 ;

architecture ISC_MUX_4 of MUX_4 is
    signal SELECCION: std_logic_vector (1 downto 0);
begin
    SELECCION <= S1 & S0;
    process (SELECCION, C0, C1, C2, C3)
    begin
        if SELECCION = "00" then Y <= C0;
        elsif SELECCION = "01" then Y <= C1;
        elsif SELECCION = "10" then Y <= C2;
        else Y <= C3;
        end if;
    end process;
end ISC_MUX_4;
```

Usando instrucciones secuenciales múltiples

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;

entity MUX_4 is
    port ( C0, C1, C2, C3: in std_logic;
           S: in std_logic_vector (1 downto 0);
           Y : out std_logic);
end MUX_4 ;

architecture ISM_MUX_4 of MUX_4 is
    signal SELECCION: integer range 0 to 3;
begin
    SELECCION <= conv_integer (S);
    process (SELECCION, C0, C1, C2, C3)
    begin
        case SELECCION is
            when 0 => Y <= C0;
            when 1 => Y <= C1;
            when 2 => Y <= C2;
            when others => Y <= C3;
        end case;
    end process;
end ISM_MUX_4;
```

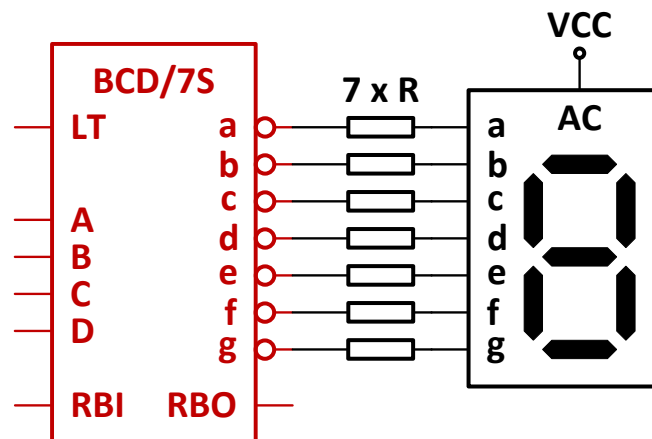
Tema 4. Bloques funcionales combinacionales

Lenguaje VHDL (Ejercicio)

Modelar en VHDL un **convertor de BCD/7 segmentos** para visualizadores de ánodo común (salidas activas a nivel bajo) que disponga, además, de las siguientes líneas:

- Una entrada **LT** (*lamp test*) que cuando reciba un nivel alto provoque el encendido de todos los segmentos del visualizador para comprobar que no están defectuosos.
- Una entrada **RBI** que cuando reciba un nivel alto, si el valor de la entrada BCD (DCBA) es "0000", provoque el apagado del display.
- Una salida **RBO** que adopte un nivel alto cuando **RBI = '1'** y **DCBA = "0000"**, simultáneamente.

La función de estas dos últimas líneas es evitar la visualización de los ceros no significativos.



Tema 4. Bloques funcionales combinacionales

Lenguaje VHDL (Consideraciones sobre VHDL)

- **Las instrucciones concurrentes deben ser completas:** Es necesario especificar qué valor debe asignarse a la señal cuando no se cumple ninguna de las condiciones. Por tanto, la instrucción **when** debe incluir siempre un **else** después de la última condición, mientras que la instrucción **with** debe finalizar obligatoriamente con **when others**.
- **La instrucción secuencial múltiple **case** también debe ser completa:** En consecuencia, es necesario indicar los valores a asignar a las señales, si la señal de referencia no posee ninguno de los valores especificados, después de **when others =>**.
- **En las instrucciones secuenciales **if** y **case**, se pueden realizar asignaciones a varias señales a la vez:** Esto se realiza como sigue.

```
SEÑAL_1 <= ____; SEÑAL_2 <= ____; ...; SEÑAL_N <= ____;
```

- **En las instrucciones múltiples **with** y **case** se puede hacer referencia simultáneamente a varios valores de la señal de referencia:** Para ello, los diferentes valores deben separarse mediante el símbolo «|»:

```
with SEÑAL_DE_REFERENCIA select
```

```
    SEÑAL <= ____ when Valor_señal_de_referencia_1 | Valor_señal_de_referencia_2 | ... | Valor_señal_de_referencia_n;
```

```
case SEÑAL_DE_REFERENCIA is
```

```
    when Valor_señal_de_referencia_1 | Valor_señal_de_referencia_2 | ... | Valor_señal_de_referencia_n => SEÑAL <= ____;
```

- **Mediante la palabra **constant** se declaran las constantes y mediante el operador «:=» se asignan valores a las mismas:** Las constantes se emplean para representar los valores fijos a asignar a las señales mediante nombres simbólicos. Estas constantes deben declararse dentro de la arquitectura, es decir después de la palabra **architecture**, pero antes del **begin** de la misma.

```
constant SIETE : std_logic_vector (3 downto 0) := "0111";
```

```
...
```

```
SEÑAL <= SIETE;
```

```
constant LONGITUD_VECTOR : integer := 8;
```

```
signal VECTOR : std_logic_vector (LONGITUD_VECTOR - 1 downto 0);
```

Tema 4. Bloques funcionales combinacionales

Lenguaje VHDL (Consideraciones sobre VHDL)

- Mediante la palabra **generic** se declaran los parámetros y mediante el operador «:=» se asignan valores a los mismos: Los parámetros se emplean para la representación de los valores fijos a emplear en el código mediante nombres simbólicos. Estos parámetros deben definirse dentro de la declaración de la entidad, antes de los puertos.

```
entity NOMBRE_DE_LA_ENTIDAD is
    generic (PARÁMETRO : Tipo del parámetro := Valor del parámetro);
    port ( ...
        );
end NOMBRE_DE_LA_ENTIDAD;
```

- Se puede acceder a una señal individual de un vector o a una parte de éste:

```
signal A : std_logic_vector (15 downto 0);
signal B : std_logic_vector (7 downto 0);
signal C : std_logic;
signal D, E : std_logic_vector (3 downto 0);
...
B <= A (15 downto 8);
C <= B (6);
D <= (3 => '1', 1 => '1', others => '0');
E <= (others => '0'); -- Equivale a E <= "0000".
```

- Mediante la palabra **array** se puede definir una matriz: Una matriz es un conjunto ordenado y numerado de vectores.

```
type MATRIZ is array (1 to 128) of std_logic_vector (7 downto 0); -- Matriz con 128 vectores de 8 bits cada uno.
signal A : MATRIZ;
begin
A (34) <= "01110100";
A (67) (7) <= '1';
```

Tema 4. Bloques funcionales combinacionales

Lenguaje VHDL (Consideraciones sobre VHDL)

- Se pueden expresar cantidades en el sistema hexadecimal usando la notación 16#Valor hexadecimal#:

```
SEÑAL <= '1' when DIRECCION = 16#FFC0# else '0';
```

- Mediante la palabra **variable** se declaran las variables internas y mediante el operador «:=» se asignan valores a las mismas: Las variables se declaran en el interior de los procesos, después de **process** y antes del **begin**. **Las variables cambian de valor durante la ejecución del proceso** en cuanto se ejecuta alguna asignación sobre las mismas, al contrario que las señales, que no adoptan los nuevos valores hasta que la ejecución del proceso ha finalizado. Las asignaciones de valores a las variables se realizan usando el operador «:=», en vez del operador «<=», como ocurre con las señales.

Ejemplo con definición de P como señal

```
architecture EJEMPLO_S of ENTIDAD is
  signal A, B, C, D: std_logic_vector (3 downto 0);
  signal P: std_logic_vector (3 downto 0);
begin
  process (A, B)
  begin
    P <= A or B;
    C <= P;
    P <= A and B;
    D <= P;
  end process;
```

Ejemplo con definición de P como variable

```
architecture EJEMPLO_V of ENTIDAD is
  signal A, B, C, D: std_logic_vector (3 downto 0);
begin
  process (A, B)
    variable P: std_logic_vector (3 downto 0);
  begin
    P := A or B;
    C <= P;
    P := A and B;
    D <= P;
  end process;
```

En el ejemplo de la izquierda, tras la ejecución del proceso, las señales C y D poseen el mismo valor (el que poseía P al inicio del proceso), ya que a la señal P no se le asigna el nuevo valor (**A and B**) hasta que la ejecución del proceso ha finalizado.

Por el contrario, tras la ejecución del ejemplo de la derecha, como la asignación de valores a la variable P es inmediata, la señal C poseerá el valor **A or B**, mientras que la señal D adoptará el valor **A and B**.

Tema 4. Bloques funcionales combinacionales

Lenguaje VHDL (Otros recursos de VHDL)

- **Definición de salidas triestado:** Para la configuración de señales triestado se emplea el valor 'Z'.

```
SALIDAS_TRIESTADO: out std_logic_vector (3 downto 0);  
...  
SALIDAS_TRIESTADO <= Valor when EN = '1' else "ZZZZ";  
-- Otra opción.  
SALIDAS_TRIESTADO <= Valor when EN = '1' else (others => 'Z');
```

- **Bucles for:** Un bucle for se ejecuta un número determinado de veces especificado por el rango de valores de un índice. Debe definirse siempre dentro de un proceso, y para ello se utiliza el formato que se muestra a continuación.

```
IDENTIFICADOR (Opcional) : for INDICE in Valor_inicial_índice to Valor_final_índice loop  
    Instrucciones del bucle.  
end loop;
```

Ejemplo: Definición de un decodificador de 6 a 64 líneas con salidas directas.

```
signal SELECCION : std_logic_vector (5 downto 0);  
signal SELECCION_INT: integer range 0 to 63;  
signal SALIDAS: std_logic_vector (0 to 63);  
  
begin  
    SELECCION_INT <= conv_integer (SELECCION);  
    process (SELECCION_INT)  
    begin  
        for INDICE in 0 to 63 loop -- Otra opción: for INDICE in SALIDA'range loop  
            if INDICE = SELECCION_INT then SALIDAS (INDICE) <= '1';  
            else SALIDAS (INDICE) <= '0';  
            end if;  
        end loop;  
    end process;
```

Tema 4. Bloques funcionales combinacionales

Lenguaje VHDL (Otros recursos de VHDL)

- **Bucles while:** Un bucle **while** se ejecuta mientras se cumpla la condición especificada en el mismo. Al igual que los bucles for, los bucles while sólo pueden definirse en el interior de un proceso. Su formato es el siguiente.

IDENTIFICADOR (Opcional) : **while** Condición **loop**
Instrucciones del bucle.
end loop;

Ejemplo: Inicialización a '0' de los 8 bits de menos peso del vector ENTRADA.

```
process (Lista de sensibilidades)
  variable INDICE : integer := 0;
begin
  BUCLE: while INDICE < 8 loop
    ENTRADA (INDICE) <= '0';
    INDICE := INDICE + 1;
  end loop;
end process;
```

- **Sentencias next y exit:** Las sentencias **next** y **exit** sólo pueden utilizarse en el interior de un bucle. La sentencia **next** interrumpe la ejecución de la iteración en curso del bucle e inicia la ejecución de la siguiente iteración del mismo. La sentencia **exit** provoca la detención inmediata de la ejecución del bucle y la salida del mismo. En el caso de existir varios bucles anidados, la sentencia **exit** provoca la salida sólo del bucle donde está ubicada dicha instrucción.